

Integración de Google Places API — Ubicación de músicos

¿Qué se ha hecho?

Se ha integrado el autocompletado de Google Places en los formularios de **crear** y **editar** perfil de músico. Cuando el usuario empieza a escribir una ciudad o región en el campo "Ubicación", aparece un desplegable con sugerencias de Google. Al seleccionar una, el campo se rellena con el nombre normalizado (ej: "Murcia, España") y se guardan automáticamente las coordenadas geográficas (latitud y longitud) en la base de datos, lo que permitirá implementar filtros de búsqueda por proximidad en el futuro.

Archivos creados o modificados

Nuevo archivo

- **assets/controllers/google-places-autocomplete_controller.js** Controller Stimulus que gestiona toda la lógica del autocomplete. Carga el script de Google Maps dinámicamente, inicializa el autocomplete sobre el campo de ubicación y, al seleccionar un lugar, rellena el texto del campo y los campos ocultos de latitud/longitud.

Archivos modificados

- **src/Entity/Musico.php** Se añaden dos propiedades nuevas a la entidad:
 - **latitud** (?float, nullable)
 - **longitud** (?float, nullable) Con sus respectivos getters y setters.
- **src/Form/MusicoType.php** Se añaden dos campos **HiddenType** al formulario (**latitud** y **longitud**) para que Symfony los reciba y persista al guardar el formulario.
- **templates/musico/new.html.twig** El bloque del campo "Ubicación" ahora tiene **data-controller="google-places-autocomplete"** y **data-api-key-value** para activar el controller Stimulus. También se renderizan los dos campos ocultos de coordenadas.
- **templates/musico/edit.html.twig** Mismo cambio que en **new.html.twig**.

Nueva migración

- **migrations/Version20260514120000.php** Añade las columnas **latitud** y **longitud** (DOUBLE PRECISION, nullable) a la tabla **musico**.
-

Qué deben hacer los compañeros

1. Obtener la rama

```
git fetch origin
git checkout feature/ubicacion-api-google
```

2. Ejecutar la migración

Este es el único paso obligatorio. Sin él, la aplicación fallará al intentar guardar un perfil porque la entidad esperará columnas que no existen en la base de datos.

```
php bin/console doctrine:migrations:migrate
```

Confirmar con **yes** cuando lo pida. Si trabajáis con Docker, ejecutadlo dentro del contenedor PHP:

```
docker exec -it <nombre-contenedor-php> php bin/console
doctrine:migrations:migrate
```

3. Limpiar la caché (recomendado)

```
php bin/console cache:clear
```

4. Verificar que funciona

1. Ir a **Crear perfil de músico** o **Editar perfil**.
2. Hacer clic en el campo "Ubicación" y escribir el nombre de una ciudad.
3. Debería aparecer el desplegable de Google con sugerencias.
4. Al seleccionar una, el campo se rellena con el formato "**Ciudad, País**".
5. Al guardar el formulario, los valores de latitud y longitud quedan almacenados en base de datos (visibles en la tabla **musico**).

Sin pasos adicionales de frontend

El proyecto usa **AssetMapper** (no Webpack ni npm). Symfony detecta automáticamente el nuevo controller Stimulus porque sigue la convención `*_controller.js` dentro de `assets/controllers/`. No hace falta ejecutar ningún comando de compilación.

Nota sobre la API key

La clave de Google Places está hardcodeada en las plantillas Twig. Es una clave de tipo *browser key* (de uso en cliente), por lo que su presencia en el HTML es inevitable con la Maps JavaScript API. Para evitar usos no autorizados, la persona responsable del proyecto debería **restringir la clave en Google Cloud Console** para

que solo funcione desde los dominios autorizados (localhost en desarrollo y el dominio de producción cuando exista).

Uso futuro: filtros por proximidad

Con **latitud** y **longitud** ya almacenados en la base de datos, se puede implementar un filtro del tipo "músicos a menos de X km" usando la **fórmula de Haversine** en un método del repositorio **MusicoRepository**. Ejemplo de consulta DQL:

```
// MusicoRepository.php
public function findByProximidad(float $lat, float $lng, int $radioKm): array
{
    return $this->createQueryBuilder('m')
        ->where('(6371 * acos(
            cos(radians(:lat)) * cos(radians(m.latitud)) *
            cos(radians(m.longitud) - radians(:lng)) +
            sin(radians(:lat)) * sin(radians(m.latitud))
        )) < :radio')
        ->setParameter('lat', $lat)
        ->setParameter('lng', $lng)
        ->setParameter('radio', $radioKm)
        ->getQuery()
        ->getResult();
}
```

Esto no requiere ninguna librería adicional ni cambios en la base de datos.